Estructuras de Datos – UNQ

Parcial – Competencia de Participantes

Descripción

El objetivo de esta evaluación es modelar un sistema de competencias entre participantes. Cada participante conoce ciertos juegos, y se busca gestionar esta información para poder realizar consultas e inserciones de manera eficiente, además de identificar ganadores potenciales según los juegos que conocen.

Para ello, se define el TAD `Competition`, que representa una competencia donde se pueden ingresar participantes y juegos, y permite obtener distintos tipos de información sobre los participantes y su desempeño en la competencia. La representación concreta está definida como:

```
data Competicion = ConsC (Map Nombre Participante)     (Map Juego [Nombre])
                         (PriorityQueue Participante) Int
```

Esta representación utiliza:

- Un **`Map`** que relaciona nombres con el participante de ese nombre.
- Un **`Map`** que relaciona una lista de nombres con un juego que dichos participantes saben jugar.
- Una **`PriorityQueue`** que posee a todos los participantes de la competición, y que permite obtenerlos de forma eficiente de mayor a menor en base a la cantidad de juegos que conocen.
- Un **`Int`** que indica la cantidad de juegos diferentes en la competición.

Ejercicios

Ejercicio 1) Dar los invariantes de representación necesarios según la descripción de la estructura.

Ejercicio 2) Implementar la siguiente interfaz de **`Competicion`**, utilizando la representación dada, y con los costos solicitados. No olvidar de calcular el costo de cada subtarea que se defina.

a. **`comenzarCompeticion :: Competicion`**, que describe una competición sin participantes ni juegos asociados. Eficiencia: O(1).

b. **`cantJuegosIngresados :: Competicion -> Int`**, que describe la cantidad de juegos ingresados en la competición dada. Eficiencia: O(1).

c. **`juegosIngresados :: Competicion -> [Juego]`**, que describe los juegos ingresados en la competición dada. Eficiencia: O(N).

d. **`nombresIngresados :: Competicion -> [Nombre]`**, que describe los nombres de participantes ingresados en la competición dada. Eficiencia: O(N).

e. **`hayClaroGanador :: Competicion -> Bool`**, que indica si la competición dada posee al menos un participante con 5 juegos sabidos. Eficiencia: O(1).

f. **`unClaroGanador :: Competicion -> Participante`**, que describe a un participante que posee al menos 5 juegos sabidos. Eficiencia: O(1).

g. **`sabeAlMenosUno :: Nombre -> Competicion -> Bool`**, que indica si el participante dado sabe al menos un juego. Nota: el participante puede no existir. Eficiencia: O(logN).

h. **`losQueEstanAlHorno :: Competicion -> [Participante]`**, que describe a los participantes que no saben jugar a ningún juego. Eficiencia: O(N logN).

i. **`ingresarParticipantes :: [Nombre] -> Competicion -> Competicion`**, que describe la competición que resulta de ingresar a participantes nuevos a la competición (mediante sus nombres), que aún no conocen ningún juego. Precondición: las participantes no existen en la competición y no hay nombres repetidos. Eficiencia: O(N logN).

j. **`ingresarJuego :: Juego -> Nombre -> Competicion -> Competicion`**, que describe la competición resultante de asociar el juego dado al participante dado en la competición dada. Precondición: el participante existe y no tiene ese juego asociado. Nota: el juego puede que no haya sido ingresado. Eficiencia: O(N logN).

Ejercicio 3) Implementar como usuario del tipo **`Competicion`**, indicando el costo obtenido.

a. **`comenzarConParticipantes :: [Nombre] -> Competicion`**, que describe una competición nueva con una lista de nombres sin juegos asociados.

b. **`todosMalos :: Competicion -> Bool`**, que indica si todos los participantes en la competición no saben jugar a nada.

c. **`losClarosGanadores :: Competicion -> [Participante]`**, describe a todos los participantes que saben jugar al menos a 5 juegos.

Ejercicio 4) Dar una posible representación para el tipo **`Participante`**, de manera de que se pueda cumplir con el orden dado para cada operación de la interfaz, pero sin implementarlas.

Anexo de interfaces

`Participante`:

```
crearP       :: Nombre -> Participante                          O(1)
nombre       :: Participante -> Nombre                          O(1)
juego        :: Participante -> [Juego]                         O(1)
cantJuegos   :: Participante -> Int                             O(1)
agregarJuego :: Juego -> Participante -> Participante           O(1)
```